Assignment 2

Exercise 2.1

Consider the "Counter" program seen in the labs and provided on ILIAS with different synchronization mechanisms used to protect a shared integer. As a reminder, the program takes as arguments two integers, T and N, and forks T threads that modify a shared integer as follows. Even threads (i.e., threads 0, 2, 4...) increment the integer N times while odd threads (i.e., threads 1, 3, 5...) decrement it N times. The program prints the final value of the integer (which should be 0 if T is even, or N otherwise).

Execute the program with N=10'000'000 and T={2,4,8,16} and all provided synchronization mechanisms. Report execution times.

Exercise 2.2

1. Read-write lock

A read-write lock allows either a single writer or multiple readers to execute in a critical section. Provide an implementation of a read-write lock in Java. You can use synchronized methods and the wait/notify mechanism if you wish. The class should provide the four methods `lockRead()`, `unlockRead()`, `lockWrite()`, and `unlockWrite()`. This implementation does not need to be FIFO, starvation-free, nor reentrant.

HINT: you might want to keep track of the number of readers and writers.

2. Starvation-free read-write lock

Try to make the read-write lock starvation-free for writers (a writer cannot be blocked forever by readers continuously requesting and acquiring the lock).

3. FIFO and reentrant read-write lock (optional)

Try to make the read-write lock FIFO and reentrant.

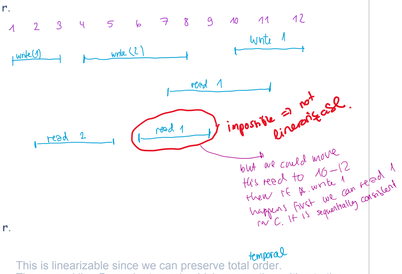
Exercise 2.3

Are the following histories linearizable or sequentially consistent? Explain your answers and write the equivalent linearizable/sequential consistent histories where applicable.

1. Read/write register

a. Concurrent threads A, B, C, register r.

```
A: r.write(1) 1
C: r.read() 4
A: r:void 2
A: r.write(2) 3
C: r:2 5
C: r.read() 6
B: r.read() 10
A: r:void 6
C: r:1 11
A: r.write(1) 7
B: r:1 12
A: r:void 9
```



Not Linearizable!
Because C:r.read will return 2, the C read must be placed after A write (2)
We cannot preserve original order. However we it is sequentially Consistent since we can separate the thread, keep their order and achieve a correct read/write order history as shown.

b. Concurrent threads A, B, C, register r.

```
A: r.write(1) 1
B: r.read() 3
A: r:void 2
A: r.write(2) 5
A: r:void 6
A: r.write(1) 8
B: r:1 4
C: r.read() 7
A: r:void 9
C: r:2 10
```

temporal

This is linearizable since we can preserve total order.
The second line B. read returns 1, which means the writing to the register has happened during the interval of line 1 to 3.
C reads and returns 2, which can be lineraized after the A write (2) runs void.
C reads 2, then A writes 1.
This is automtically also sequentially consistent.

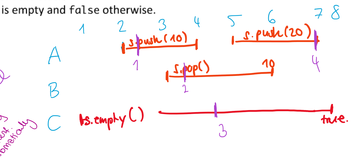
2. Stack

We have the following operations:

- push(x) pushes element x on the stack, returns `void`;
- pop() retrieves an element from the stack;
- empty() returns `true` if stack is empty and `false` otherwise.

a. Concurrent threads A, B, C, stack s.

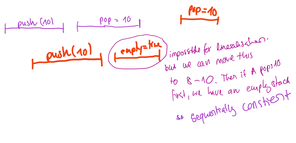
```
C: s.empty()
A: s.push(10)
B: s.pop()
A: s:void
A: s.push(20)
B: s:10
A: s:void
C: s:true
```



linearizable
⇒ sequentially consistent automatically

b. Concurrent threads A, B, C, stack s.

```
A: s.push(10)
B: s.push(10)
A: s:void
A: s.pop()
B: s:void
B: s.empty()
A: s:10
B: s:true
A: s.pop()
A: s:10
```



impossible for linearizability. but we can move this to 8-10. Then if A pop 10 first, we have an empty stack ⇒ sequentially consistent

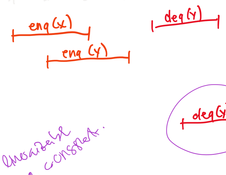
3. Queue

We have the following operations:

- enq(x) inserts element x in the queue, returns `void`;
- deq() retrieves an element from the queue.

a. Concurrent threads A, B, C, queue q.

```
A: q.enq(x)
B: q.enq(y)
A: q:void
B: q:void
A: q.deq()
C: q.deq()
A: q:y
C: q:y
```



not linearizable
not seq. consistent.

we cannot dequeue y twice. This history is impossible to happen